

Technical University of Košice
Faculty of Mechanical Engineering

PROJECT REPORT

Development of a PID-controlled propeller setup. Mathematical modeling of a system behaviour

Author: Artem Zhernovyi
Student of Industrial Mechatronics

Košice — 2025

Abstract

The work investigates the control of a DC motor using an Arduino microcontroller. The aim is to develop a feedback system model with a PID controller for thrust stabilization. The system was simulated in Python using experimentally obtained data, which allowed for an in-depth analysis of its performance. The results make it possible to determine the optimal configuration of the PID parameters to ensure stable and efficient system operation.

Keywords: PID controller, DC motor, Arduino, feedback system, modeling

Contents

1 Introduction **3**

2 System description **4**
- 2.1 Constructional description of the propelle setup . . . 4
- 2.2 Electrical parts of the setup . . . 5
- 2.3 Electrical description of the control block . . . 5
- 2.4 Constructional description of the control block . . . 7
- 2.5 Software description of the setup . . . 8
- 2.6 Conclusions of the chapter . . . 9

3 Work of the system and data collection **10**
- 3.1 Description of each term of PID controller and it's role . . . 10
  - 3.1.1 P-parameter . . . 10
  - 3.1.2 I-parameter . . . 11
  - 3.1.3 D-parameter . . . 13
- 3.2 Tuning of the system and analysis of it's behavior . . . 13
- 3.3 The description of the PID controller tuning video . . . 14
- 3.4 Conclusions of the chapter . . . 15

4 Experimental data of the DC motor **16**
- 4.1 Description of measurement process . . . 16
- 4.2 Logical description and measurement procedure . . . 16
- 4.3 Analysis of experimental data . . . 18
- 4.4 Conclusions of the chapter . . . 19

5 Mathematical model of the system **20**
- 5.1 Mathematical setup of the model . . . 20
- 5.2 Analysis of the model . . . 25
- 5.3 Conclusions of the chapter . . . 27

6 Conclusions of the whole report **28**

7 References **29**

1 Introduction

The rapid development of automation and mechatronic systems has increased the need for precise and adaptive control of electric drives. Direct current (DC) motors are widely used in robotics, prosthetics, and industrial applications due to their simple structure and ease of control. However, maintaining stable thrust or speed under variable load conditions remains a challenging task that requires effective feedback control.

One of the most widely used approaches for such control problems is the Proportional–Integral–Derivative (PID) controller. Despite its conceptual simplicity, the correct tuning of PID parameters is crucial for ensuring system stability and performance. Modern microcontrollers such as Arduino provide a low-cost and flexible platform for implementing and testing PID-based control systems.

This work focuses on the development and analysis of a feedback control system for thrust stabilization of a DC motor using an Arduino microcontroller. The system was modeled and simulated using programming language Python based on experimental data to study the dynamic behavior of the motor and evaluate the performance of implemented control algorithm.

2 System description

2.1 Constructional description of the propelle setup

Propeller setup is the main part of the system, that consists of stable foundation (5) and moving part with one degree of freedom, that allows to set the electrical DC-motor (9) with a propeller (8) on the one side and counterweight (2) on the other. The presence of the counterweight, which can move along the threaded rod (1), gives the opportunity to shift the center of mass of the moving part relative to the axis of it's rotation. A clear definition of the distance between the axis of movement and the center-of-mass point of the part is important for subsequent modelling of the system. The axis is formed by the bolt (6), that is rigidly connected with the moving part and supported by bearings.
(Numbers in parentheses refer to the components shown in Figure 2.1.1.1.)

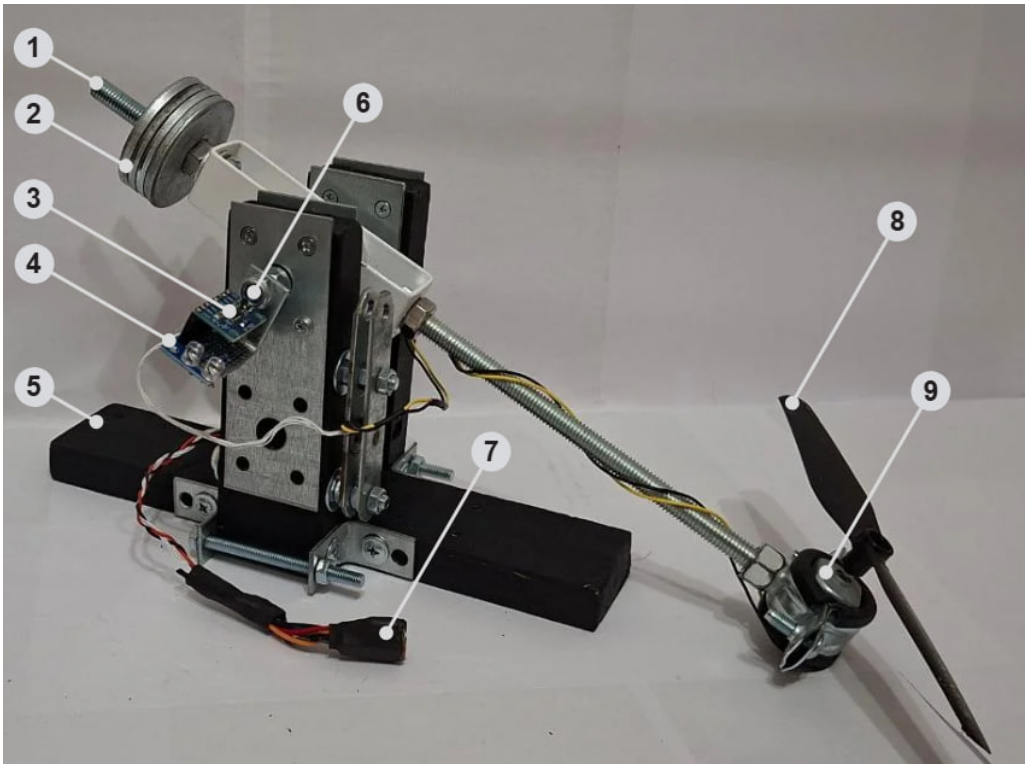


Figure 2.1.1 Propeller setup with numbered components:
1 – Threaded rod of the counterweight; **2** – Counterweight (stabilizer); **3** – Sensor (MPU6050); **4** – Moving plate (carriage for sensor); **5** – Support; **6** – bolt (axis of rotation); **7** – Electrical connector (plug type); **8** – Propeller blades; **9** – DC motor.

2.2 Electrical parts of the setup

For precise detection of the position of the moving part in space, an MPU6050 sensor (3, Figure 2.1.1) is mounted on the rotation axis. The sensor can be easily replaced in case of failure thanks to female header connector, placed on the PCB prototyping board (4, Figure 2.1.1). The sensor mount is rigidly connected to the rotation axis of the moving part (6 , Tab. 2.1.1.1) and is capable of measuring its instantaneous position in space. All electrical connections in the system are combined into a single plug (7, Tab.2.1.1), which must be connected to the control unit of the system.
The plug-in connection, used to mount the sensor, is made on PCB board, where two LEDs and female header for the sensor are soldered and connected to triangular metal part with the special holes for fixation on the rotation axis (Figure 2.2.1) Special LEDs are used for calibration of the sensor during the setup process before work.

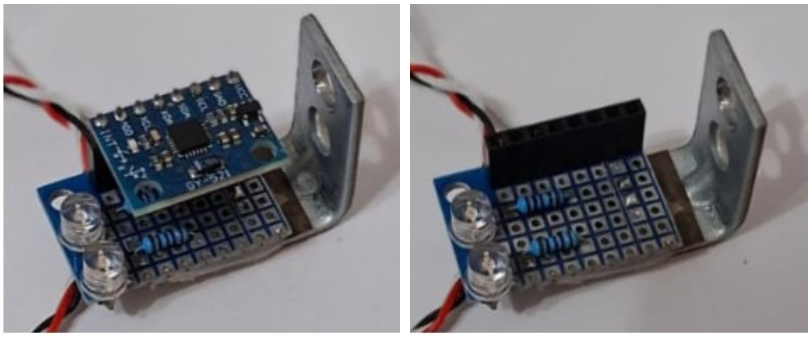


Figure 2.2.1 The plug-in connection

2.3 Electrical description of the control block



Figure 2.3.1 Control block
The control-block is an embedded into plastic block, electrical scheme, made using PCB board to provide process of setting of the parameters of PID-controoler using an encoder and LCD-display.

Electrical components of the system are placed on a PCB board, that is enclosed in a special box. Each element of the scheme is connected to the Arduino board using special input headers. The scheme of the system are added in Figure 2.3.2

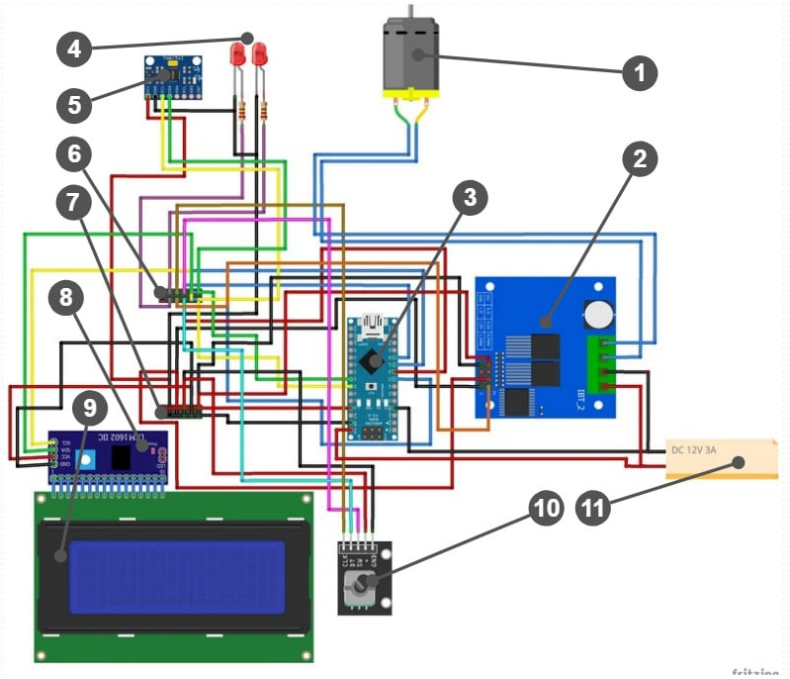


Figure 2.3.2 Electrical scheme of the system:
1 – DC motor; **2** – DC motor driver BTS7960 43A; **3** – Arduino Nano board; **4** – Calibrating sensor LEDs; **5** – MPU-6050 sensor; **6** – logical plug-in connector; **7** – power plug-in connector(+5V, GND); **8** – I2c converter for LCD display; **9** – LCD display; **10** – encoder; **11** – DC power apply (12V 3A).

Enclosing of the board in special box provides the reliability and user-friendliness of the system. Usage of the headers (6,7, Figure 2.3.2) and plug-in typed components (7, Figure 2.1.1) makes the system more unified. The image of the real PCB with electrical components is in Figure 2.3.3

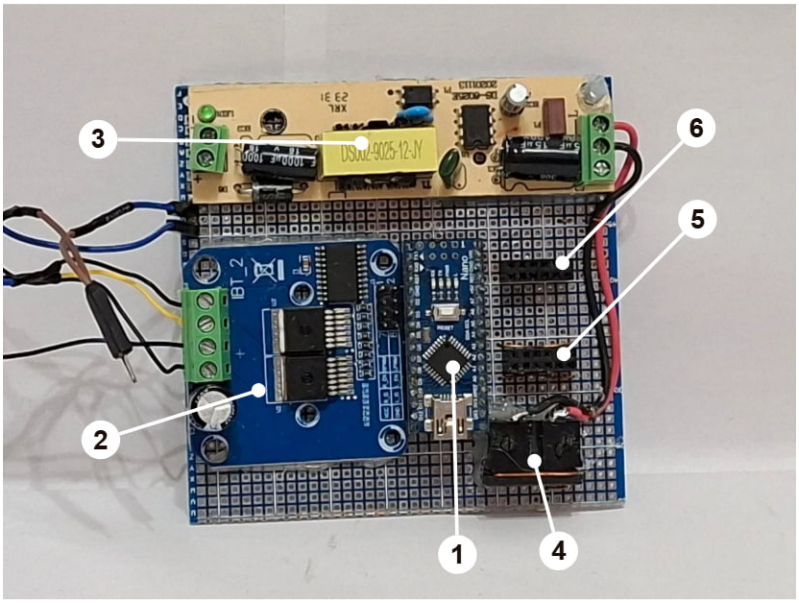


Figure 2.3.3 Assembled electrical components, embedded in the control block:
1 – Arduino Nano board; **2** – DC motor driver BTS7960 43A; **3** – DC power apply (12V 3A); **4** – AC-cabel connector; **5** – power plug-in connector; **6** – logical plug-in connector;

2.4 Constructional description of the control block

The construction of the control block is made using 3D-printed part, combined with metal plates, that are connected using screws(M8×140–8.8–Zn–ISO 4017).
The whole description of the construction is in the Figure 2.4.1

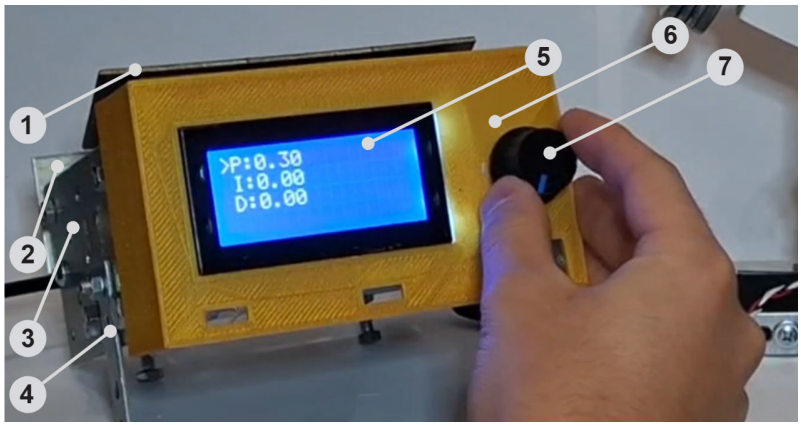


Figure 2.4.1 Constructional description of the control block:
1 – Flat connector plate 120×40×1.8; **2** – Flat connector plate 120×40×1.8; **3** – Flat connector plate 140×60×1.8; **4** – Flat connector plate 60×15×1.8; **5** – LCD display; **6** – 3D printed frame (STL file [1]) **7** – Encoder.
The control board (Figure 2.3.2) is implemented into frame (6, Figure 2.4.1) using magnets. The discription is in Figure 2.4.2

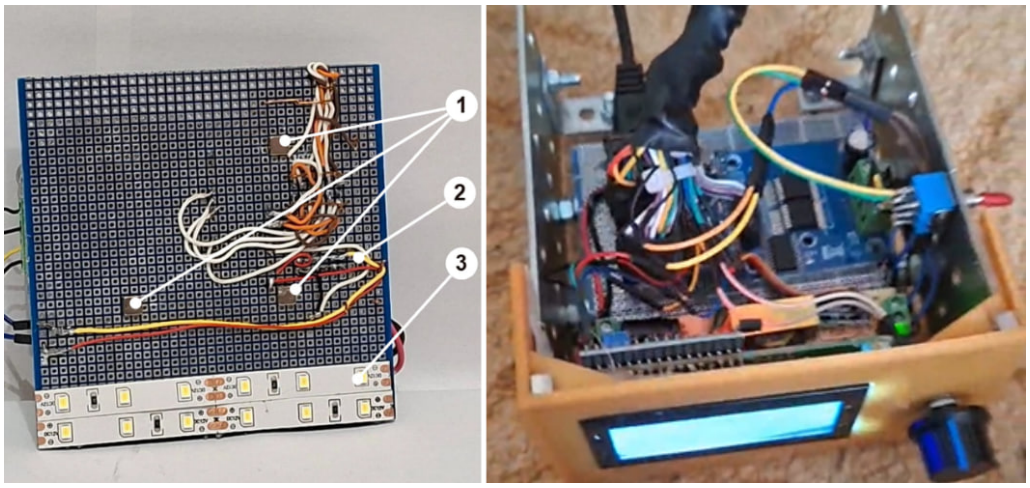


Figure 2.4.2 Implementing of the assembled electrical components in control block:
1 – magnets for connecting; **2** – connecting wires; **3** – LED strip light (can be used in addition) ;

2.5 Software description of the setup

The software of the system allows to set the P,I,D (proportional, integral, derivative) parameters dinamicaly in real time using rotation of the encoder (7, Figure 2.4.1). Choosing the parameter, that is needed to change is realized by pressing the encodder button.
Current values of each parameter are displayed on the screen (5, Figure 2.4.1).
The system gets information about each of P,I and D parameters and recalculates the value of control signal for each moment depending on changes of the system using formula (1)[2].

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\,d\tau + K_d \frac{de(t)}{dt} \tag{1}$$

Where control signal $u(t)$ is defined as the product of each component and its corresponding coefficient. The proportional term is defined as the product of the proportional gain coefficient K_p and error $e(t)$, that provides changing of $u(t)$ proportionally to the deviation of the system from the setpoint. The integral term is defined as the product of the integral gain coefficient K_i and and the integral of the error function $e(\tau)$ over time, where τ represents the time variable of integration. The last term that influences the control signal $u(t)$ is the derivative term, which is defined as the product of the derivative gain coefficient K_d and the time derivative of the error $\frac{de(t)}{dt}$
After the system calculates the value of the control signal, it is approximated to the range of 0–255 for PWM generation.
The full version of the code is attached in [3].

2.6 Conclusions of the chapter

1) Using the Arduino Nano board as the core of the system and combining it with a motor driver module BTS7960 43A, an MPU6050 sensor, and an encoder, makes it possible to build a PID controlled device.
2) The basic parameters of the system are displayed on an LCD screen connected through an I2C converter. To make the device user-friendly, 3D printing technology was used in combination with screwed metal plates. The structural design of the control box allows all electronic components to be embedded inside it.
3) Two separated parts of the whole system (propeller setup and control block) allow to achieve precise control of motor stabilization and provide data for creating a computer-modeled system.

3 Work of the system and data collection

3.1 Description of each term of PID controller and it's role

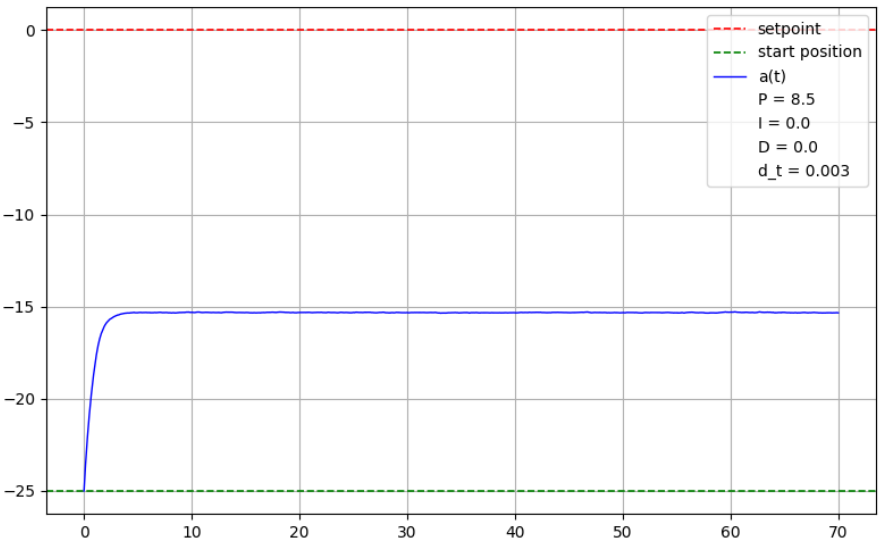
Implementing the PID controller as a reliable technical decision to control dynamic systems is caused by its capability for fast reaction to changes in the system and quick adaptation of the control signal according to requirements using properly set coefficients. "Proportional integral derivative (PID) controllers have been used for several decades in industries for process control applications... The reason for their wide-ranging popularity lies in the simplicity of design and good performance including low percentage overshoot and small settling time for slow process plants."[4]

3.1.1 P-parameter

Proportional term is the main element that directly reacts to the magnitude of the error between the setpoint and the current condition of the system. It forms an immediate reaction of the system, increasing or decreasing the signal of the system proportional to the current error.

$$u_P(t) = K_P \cdot e(t) \tag{2}$$

where
$u_P(t)$ – proportional component of the control signal,
K_P – proportional gain coefficient,
$e(t)$ – error ($error = setpoint - current\ posistion$).
"The proportional term provides an overall control action proportional to the error signal through the all-pass gain factor."[5] Using the developed mathematical model, it is possible to create a graph where a(t) represents the system position as a function of time (axis Oy), the Ox axis corresponds to time t.



The graph illustrates, that the signal starts to increase but becomes constant in certain point. The mistake is constant, the signal is constant ($K_P \cdot e(t) = \text{const}$).

3.1.2 I-parameter

The integral term enables the elimination of steady-state error. It accumulates (integrates) the error over time. It accounts not only for the current error as the proportional term does, but for the entire period of the error.

$$u_I(t) = K_I \int_0^t e(\tau)\, d\tau \tag{3}$$

where
$u_I(t)$ – integral component of the control signal,
K_I – integral gain coefficient,
$e(\tau)$ – current error.

When the control signal u is within the boundaries from 0% to 100%, the system constantly adjusts the control action according to the changes, operating in a closed loop (Figure 3.1.2.1). The system can become overregulated due to the constant accumulation of the error, which leads to saturation.

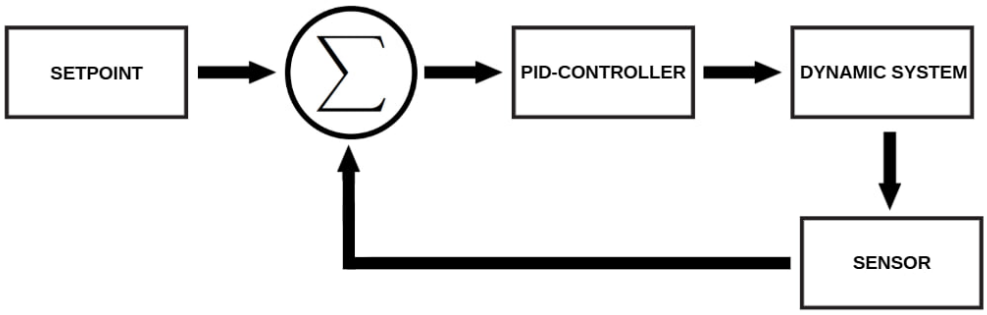
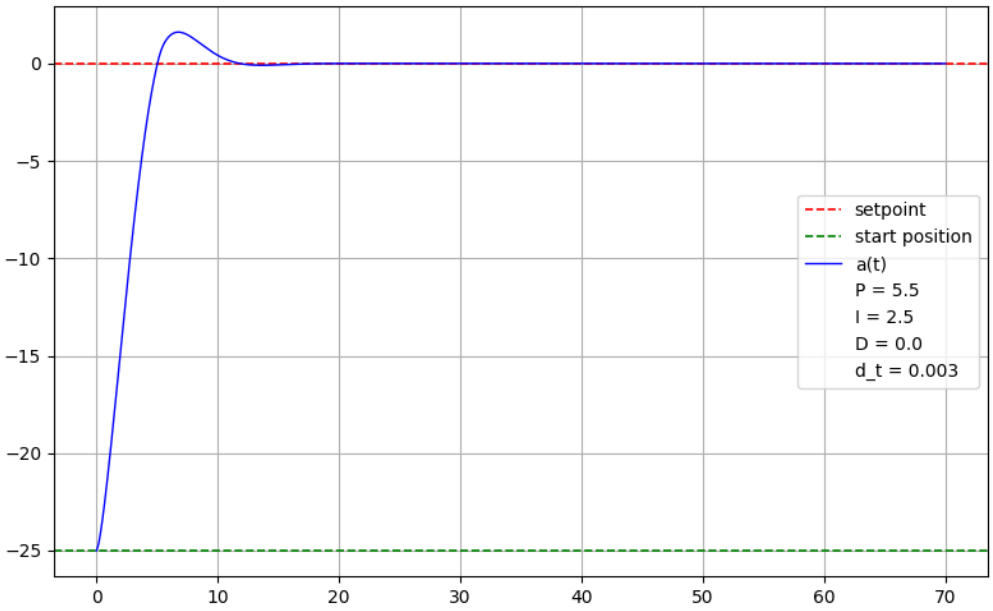
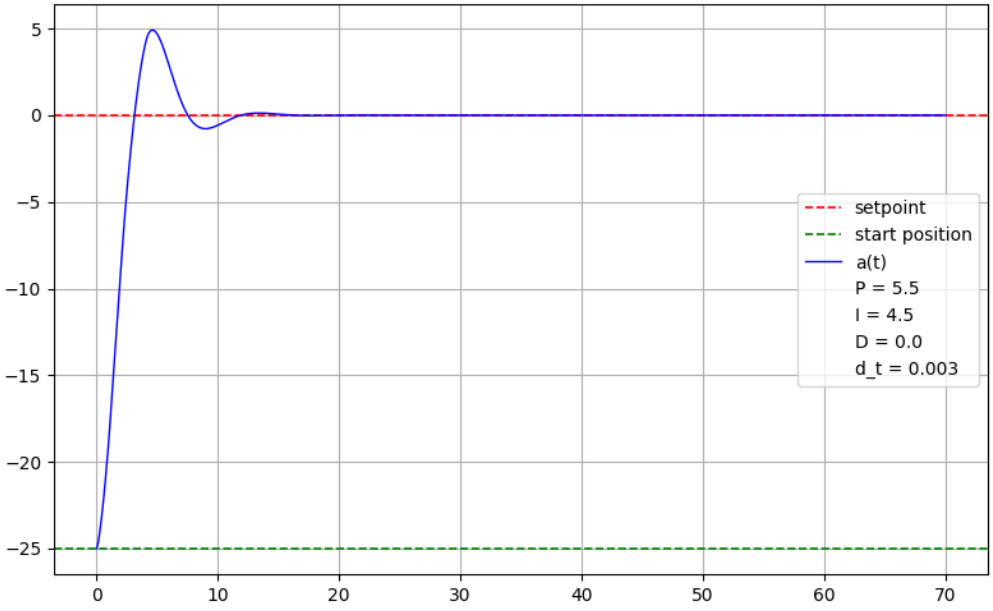


Figure 3.1.2.1 Schematic description of the closed-loop operation of the PID controller

"When the actuator saturates, the loop is effectively open... if the control is such that a long period of saturation exists, it is possible for the integral term to keep integrating even when it is not reasonable to do so... the integrator output will drift higher. This is called integrator windup."[6] Using the same mathematical model, it is possible to create the graph of the system's work and see that it needs time to increase the value of the control signal (PWM duty cycle) to achieve the set position.
Increasing of the integral term causes the system to experience overshoot and oscillations, that can be observed in the period between 4 and 10 seconds on the graph.



When the integral term gets too large - the system system responds to changes faster but the oscillations are larger, that can be observed in the period between 3 and 10 seconds on the graph.



3.1.3 D-parameter

The derivative term does not react to the magnitude of the error, but to the rate of its change. It predicts the change of the system parameters and sets the control signal that slows down this change — acting like a damper.

$$u_D(t) = K_D \frac{de(t)}{dt} \tag{4}$$

where
$u_D(t)$ – derivative component of the control signal,
K_D – derivative gain coefficient,
$e(t)$ – error.

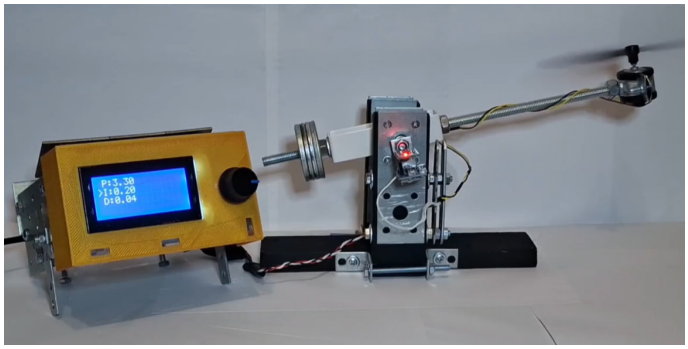
"The derivative term improves transient response through high-frequency compensation."[7]
The real system is different from the mathematical model because it includes noise in the sensor signal, friction between moving parts and inertia of mechanical components. These effects make the real system more sensitive to changes of the D parameter. In practice, the real system reacts sharply when the D value is increased, while the simulation model almost does not change the signal, since the position of the system changes only slightly.

3.2 Tuning of the system and analysis of it's behavior

The empirical method of tuning of PID controller consists of three principal steps to do:
1. The first step is to start increasing the proportional term to achieve the behavior of the system, when it oscillates with stable amplitude. That is done to achieve the stable reaction of the system to fluctuations.
2. The second term that is needed to increase is the derivative term. The term is increased until the amplitude of fluctuations of the system becomes less. A large value of the derivative term leads to sharp reaction of the system and appearing fluctuations in the control system. The aim of this term is to eliminate noise the and make the system more stable.
3. When the system becomes more stable, it is needed to set the third term — integral. The role of the integral term in the system is to eliminate the static error over time. The large value of the integral term leads to slow oscillations and destabilization of the system over time.

Term	**Aim**	**Effect of over-tuning**
Proportional	Accelerates system response, decreases error	Causes oscillations
Derivative	Attenuates oscillations	Increases noise in control signal
Integral	Eliminates steady-state error over time	Causes slow oscillations; may destabilize the system

3.3 The description of the PID controller tuning video



The **link** to the video is shown in the QR code.
The approach described in the previous section is used in practice. The video linked above shows the process of tuning the PID controlled setup. The video is divided into chapters, each of which is described as follows:

Time	Observation
0:06–0:12	The system starts to oscillate, but the value of the control signal is not high enough to increase the motor thrust.
0:12–0:18	Motor thrust increases and the position changes in the direction of the setpoint.
0:22–0:23	Increasing the derivative parameter causes a sharp increase in the control signal; the system becomes more stable.
0:24–0:30	The fluctuation amplitude does not decrease over time due to a steady-state error that cannot be eliminated by the proportional and derivative terms.
0:30–0:34	Increasing the proportional term from 3.10 to 3.10 leads to oscillations in the system.
0:34–0:40	Increasing the integral parameter stabilizes the system by eliminating the steady-state error.
0:40–0:43	Destabilization of the system and it's rapid response.

The system has a fast response to its destabilization. To clearly measure the time needed to eliminate the position error, several attempts of destabilization were shown at $t = 00{:}52, 00{:}57, 01{:}26, 01{:}33, 01{:}40, 01{:}46, 01{:}52$. The moment of uncommanded disturbance is shown at $t = 01{:}14$ and 01:20.
This behavior may be caused by system imperfections, including friction between moving parts, sensor-signal noise due to lack of cable shielding, and possible Arduino code execution delays that effectively increase the sampling interval. The reaction period is about 1.5 s, but the last attempt of destabilization (01:57) showed that the system requires 2 to 3 s to eliminate the position error.

3.4 Conclusions of the chapter

1) Implementing the PID-controller algorithm into the system leads to increased predictability of its operation and makes the control process more precise. Setting the P, I, and D parameters provides stable operation of the system. However, depending on the difference between the setpoint and the initial position, it may take different amounts of time to reach the correct control signal, which can lead to it's fluctuations.
2) The system has static low-amplitude oscillations caused by its imperfections, which make the difference between the real system and its mathematical model. This should be taken into account when comparing the output data of the model with the real behavior of the system.
3) The empirical method of system tuning only allows to approach the correct operation. In order to make the system as stable as possible, it is necessary that the parameters are precisely calculated using available methods.

4 Experimental data of the DC motor

4.1 Description of measurement process

The aim of the measurements is to collect information about the dependence of the magnitude of the voltage, electrical current and the thrust of the motor on the PWM signal, to investigate the form of the signal using an oscilloscope and analyze the quality of the signal.

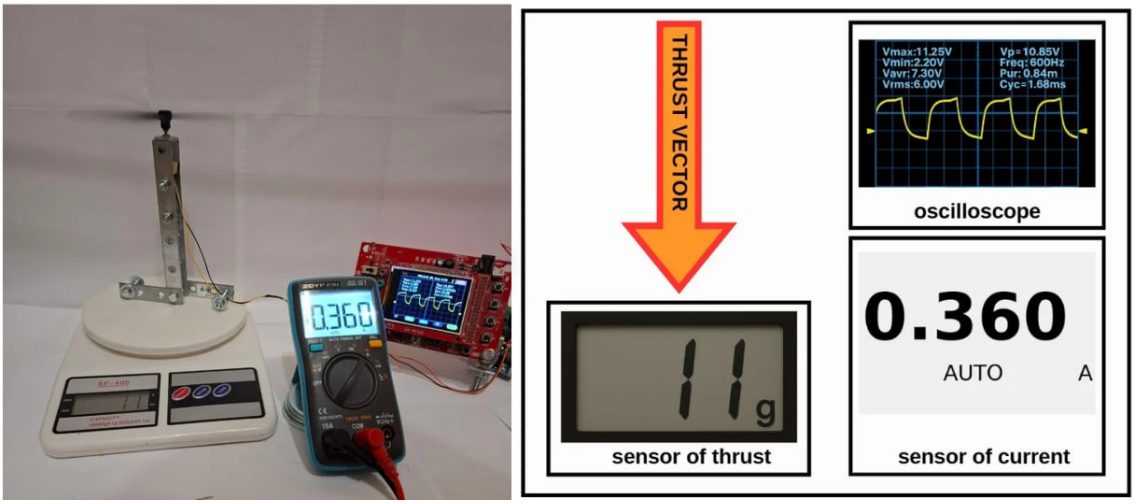


Figure 4.1.1 The description of the setup of the measurement process

The aerodynamic characteristics of the propeller are the same regardless of the rotation direction. To improve measurement accuracy, the thrust vector was directed downward so that the weight sensor correctly measured the additional load generated by the motor thrust.The Figure 4.1.1 shows the form of the PWM signal (the image of an oscilloscope screen) and it is a proof that the circuit is not ideal. The inductive characteristics of the motor and other technical features of the components, used in the circuit, create various types of noise in the measurements. A special PWM module is used in this system to provide speed control of the motor, but it is not perfect and cannot generate a pure square-wave signal.
Previous measurements have shown that the maximum voltage in the circuit is around 12 volts. The maximum thrust of the motor reaches 15 grams (0.147 newtons), and Increasing the voltage does not lead to higher thrust but causes excess energy to dissipate as heat, which results in overheating and damage to the motor.

4.2 Logical description and measurement procedure

To conduct the measurments and get importand data for modelling, the process has special sequence of actions. It's needed to measure magnitudes of voltage, electrical current in a circle and get the form of the signal for each magnitude of thrust from 2 to 15 grams. The siquance of actions and logic of the measurement are illustrated in block-scheme in Figure 4.2.1

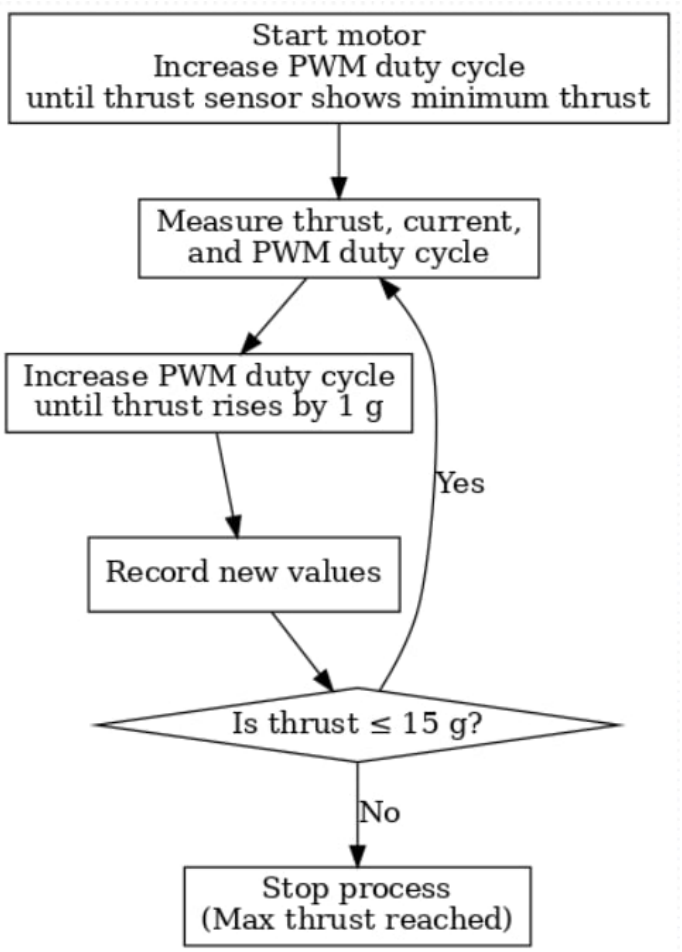


Figure 4.2.1 The block-scheme of the measurement process

The **link** to the video that contains the measurement process, based on which the data table was created, is attached in the QR code.

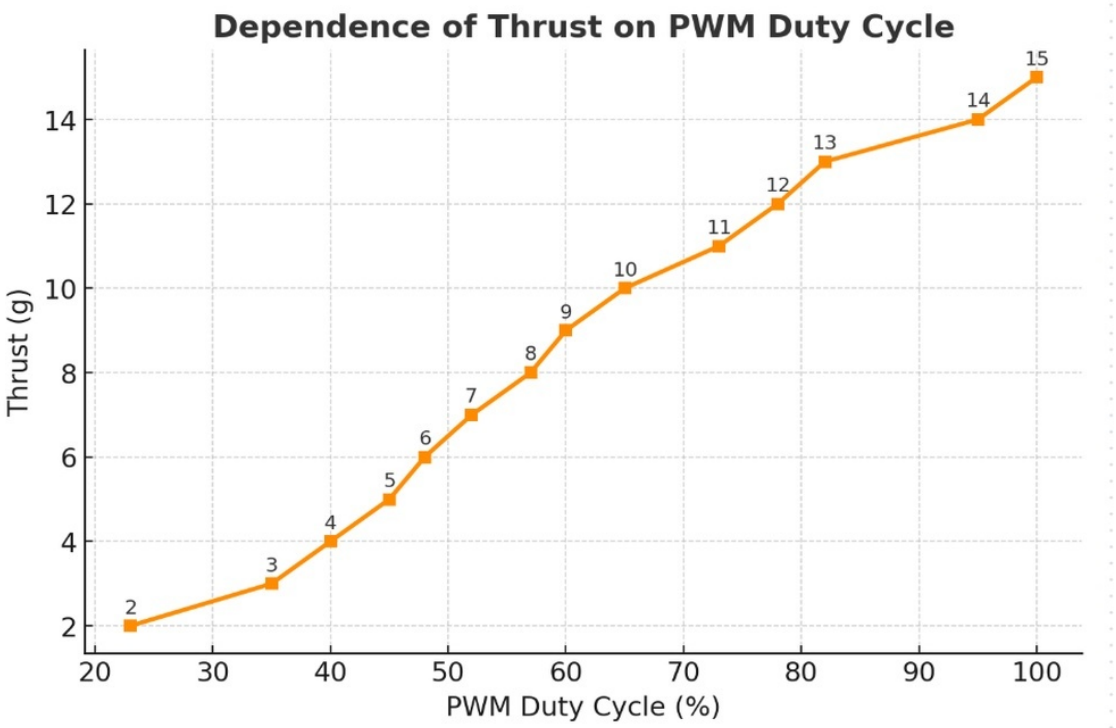
4.3 Analysis of experimental data

The results of the measurements are given in Table 4.3.1

Thrust (g)	PWM Duty Cycle (%)	Current (mA)
2	23.1	62
3	34.7	115
4	39.7	148
5	45.7	214
6	47.8	232
7	52.1	271
8	56.5	314
9	59.4	339
10	65.7	413
11	73.2	501
12	80.0	583
13	83.0	654
14	97.4	879
15	100.0	896

Table 4.3.1 The results of the measurement

Using data from table is able to build to graphs of dependense (Figure 5.3.2)



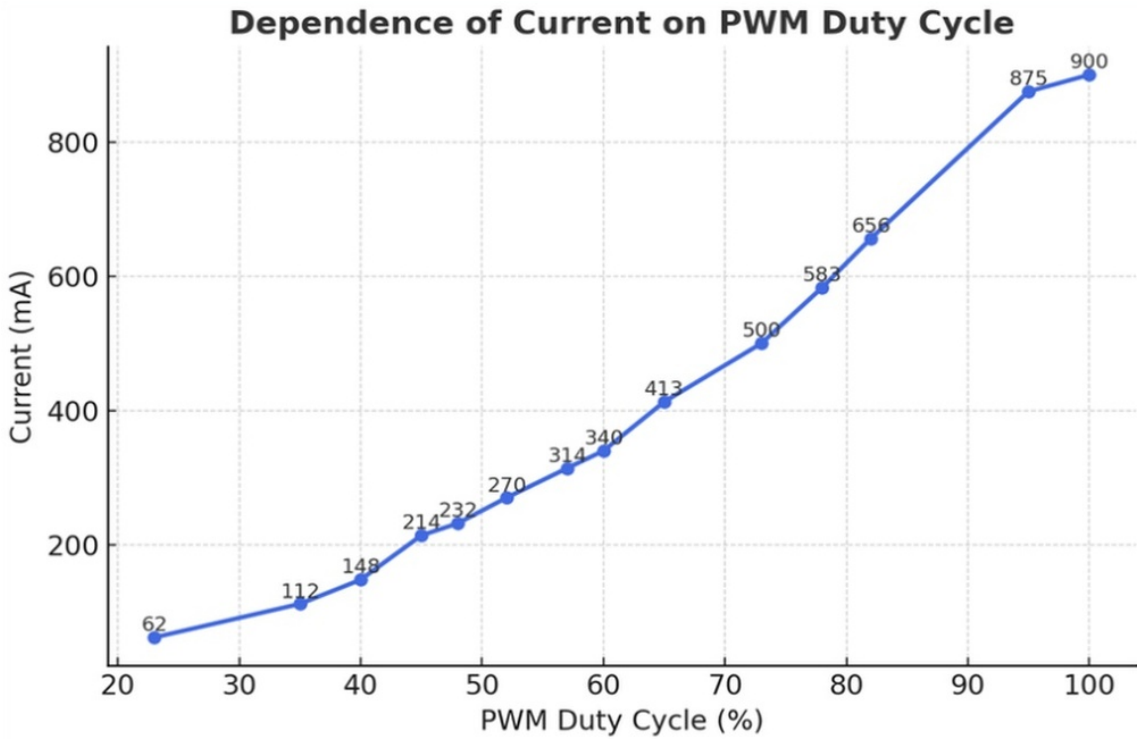


Figure 4.3.2 Graphs of variable dependencies

4.4 Conclusions of the chapter

1) Obtaining data on the dependence of thrust on the control signal makes it possible to build a mathematical model for simulation, based on the real physical characteristics of the system.
2)The imperfections of the system introduce additional noise into these data, which prevents the output results of the model from approaching the real data of the actual system.
3) The idea of including noise in the simulation to make it more precise does not fundamentally improve the accuracy of the mathematical model, since the noise in the real system is generated by multiple interdependent factors that lead to varying system behavior. Parameters such as sensor signal noise, friction between the rotation axis and the mounting, and the aerodynamic characteristics of the system make it more complex for precise modeling.

5 Mathematical model of the system

5.1 Mathematical setup of the model

To create the mathematical model of the system, it is necessary to make special description of the movement of the propeller setup. There are 3 key points that are used in the model and that are showed in Figure 5.1.1

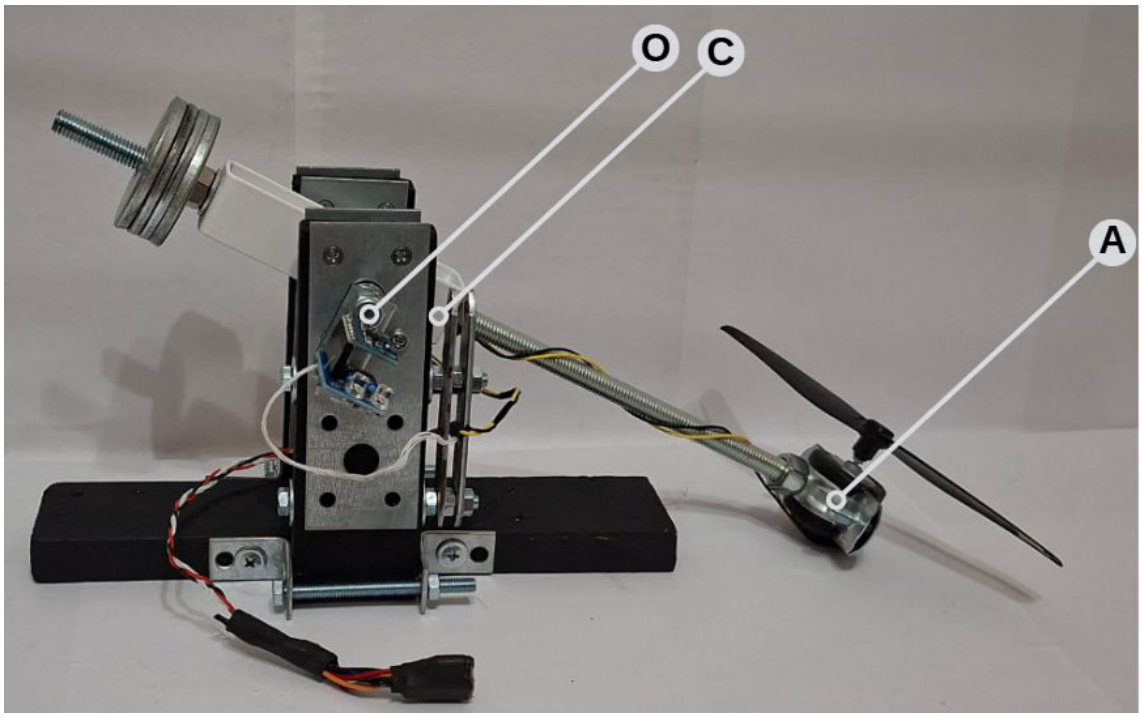


Figure 5.1.1 Definition of key points in the propeller setup for mathematical modeling
O – the axis of rotating movement; **C** – the point of mathematical centre of mass; **A** – the point of application of the thrust force;

To build an accurate model of a dynamic system, it is necessary to introduce an additional point — the center of mass (C). Knowing the position of the point C relative to the axis of rotation makes it possible to formulate the mathematical model using the laws of motion of a body with constant mass.
The schematic description of the model is given in Figure 5.1.2

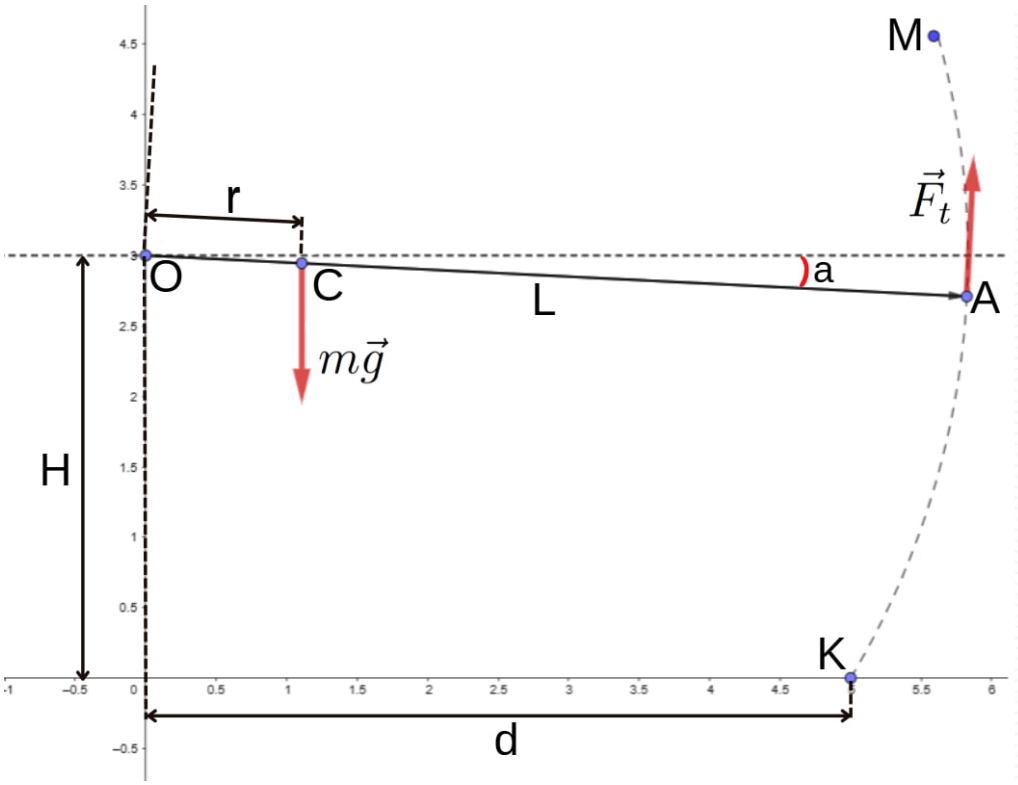


Figure 5.1.2. Schematic definition of the dynamic system

KM – the arc of the trajectory of point A's motion; $\vec{F}_t$ – the vector of thrust force of the propeller; $m\vec{g}$ – gravitational force vector (weight) acting vertically downward from the center of mass at point C; r – distance between the axis of rotation and the center of mass; L – distance between the axis of rotation and point A; H – distance between the axis of rotation and the surface where the setup stands.
To describe the movement of OA along the arc KM, is needed to define two vectors: the vector of gravitational force $m\vec{g}$ applied at point C and the vector of the motor thrust $\vec{F}_t$ applied at point A.
$m\vec{g}$ - always directed downward.
$\vec{F}_t$ - always perpendicular to OA.

The difference between the current position of point A and the setpoint is expressed by the angle α between the dashed horizontal line (setpoint) and OA. Depending on this difference, the projection of the vector $m\vec{g}$ on the O_y axis (the vertical axis) is defined as $mg\cos(\alpha)$.
Considering point C as a point moving with acceleration, the law of the lever can be applied. Formula (5) expresses this relationship.

$$F_t L = mar + mgr\cos(\alpha) \tag{5}$$

Using formula (5), equation (6) expresses the acceleration of the center of mass (point C).

$$a = \frac{F_t L - mg\cos(\alpha)}{mr}, \tag{6}$$

To describe the rotational motion of C, it is needed to define the distance OC, that is ℓ, formula of angular acceleration (7) is combined with (6).

$$\varepsilon = \frac{a}{r}, \tag{7}$$

Equation (8) describes the dependence of angular acceleration ε on the force of thrust F_t

$$\varepsilon = \frac{F_t L - mg\cos(\alpha)}{mr^2} \tag{8}$$

Using the description of angular acceleration ε and formula (9) that describes the angular velocity ω, it is possible to calculate the change of the initional positin of the system for accelerated angular motion.

$$\omega = \varepsilon t \tag{9}$$

The dependence of $\Delta\alpha$ on time is described by equation (10).

$$\Delta\alpha = \omega_0 + \omega\Delta t + \frac{\varepsilon\,\Delta t^2}{2} \tag{10}$$

To implement the physical data into model, it has to calculate the magnitude of thrust for each Δt, where Δt is sampling time of the system in PID controller.
To accomplish this task, the model has to make the linear interpolation of the magnitude of $\vec{F_t}$ using physical table of data (Table 4.3.1), that has to be implemented into the model using special arrays for the magnitudes of thrust and PWM-coefficient (the array n has been shortened for formatting purposes).

```
# Table of experimental data (PWM% -> F )
n = [0, 23.1, 34.7, 39.7, 45.7, 47.8, 52.1, 56.5 ... 97.4, 100, 101]
p = [0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

PID controller gives the magnitude of u (control signal), that has to change the thrust vector.
To calculate the thrust magnitude for each value of u, the model first finds the two adjacent PWM coefficients that bracket u (Table 4.2.1).
After the bounds $n[i] \leq u < n[i+1]$ are identified, the thrust is computed by linear interpolation using formula (11).

$$F = g \cdot 10^{-3} \left(p[i] + \frac{u - n[i]}{n[i+1] - n[i]} \right) \tag{11}$$

```
i = 0
    while True:
        if u >= n[i] and u < n[i + 1]:
            F = g * 0.001 * (p[i] + (u - n[i]) / (n[i + 1] - n[i]))
            break
        i += 1
```

Using this formula of linear interpolation, the system is able to get a new value of F_t for each Δt. Getting the value of F_t, using formulas (8), (9), and (10), it is possible to get the value of $\Delta\alpha$ at the end of the loop. Each iteration of the loop has its own value of the current position of the system because this position is updated using the value of $\Delta\alpha$.

```
    E = (F * L - m * g * l * math.cos(math.radians(a))) / (m * l**2)
    w = E_prew * d_t

    d_a = w * d_t + (E * d_t**2) / 2
```

The system collects data that were created by the model in a special array for each magnitude: position of the system in time.
Figure 5.1.1 - a block-scheme of the whole model code[8]. The names of constants in the code have been modified for better readability. The full description of the code is provided in the comments within the source file.

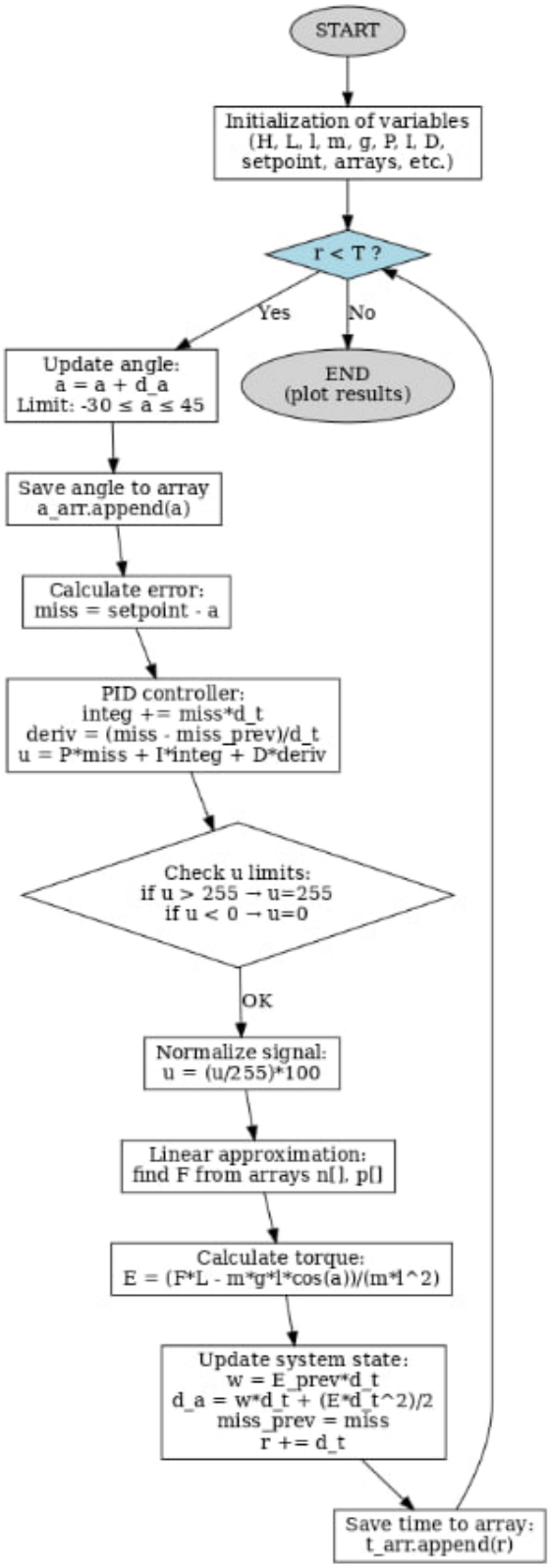


Figure 5.1.3. The block-scheme of the mathematical model

5.2 Analysis of the model

In order to simulate the process, the system must receive input data such as Table 4.3.1 and other data:

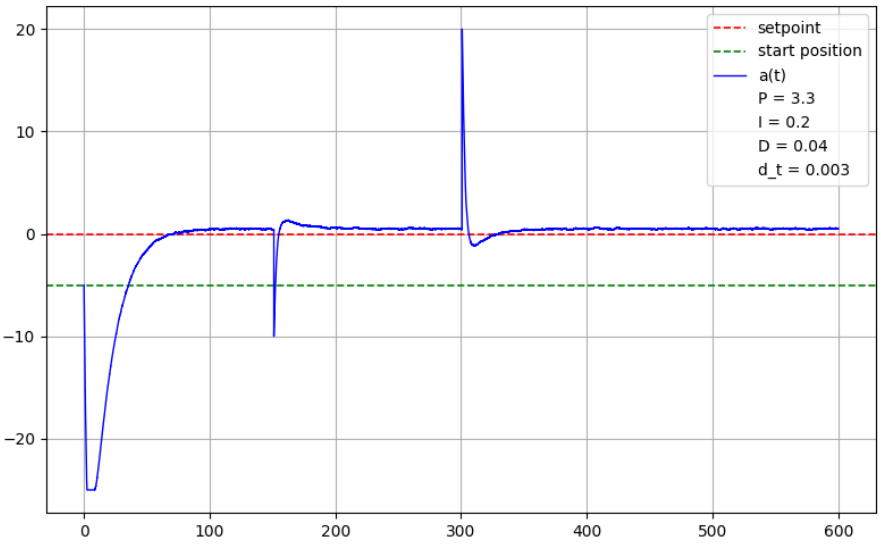
```
# Table of experimental data (PWM% -> F )
n = [0, 23.1, 34.7....97.4, 100]
p = [0, 2, 3, 4, 5....15]

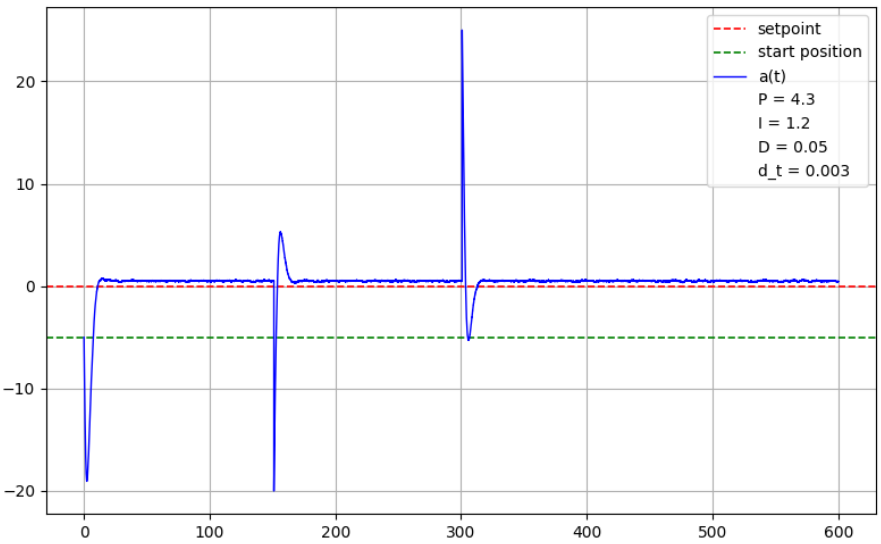
# Constants and initial model parameters (according to Figure 5.1.2)
L = 0.180
l = 0.0035
m = 0.381
g = 9.8

# PID parameters
P = 3.3
I = 2.2
D = 0.5

# Setting the noise amplitude limits
lower_limit = 0
upper_limit = 1.5
```

The graph is the result of the modeling of the system.
Modeled system needs time to achieve the setpoint and reacts on sharp changes so fast. The differences between the behavior of the real system and output data of it's model caused by the imperfection of the system, that was described above.

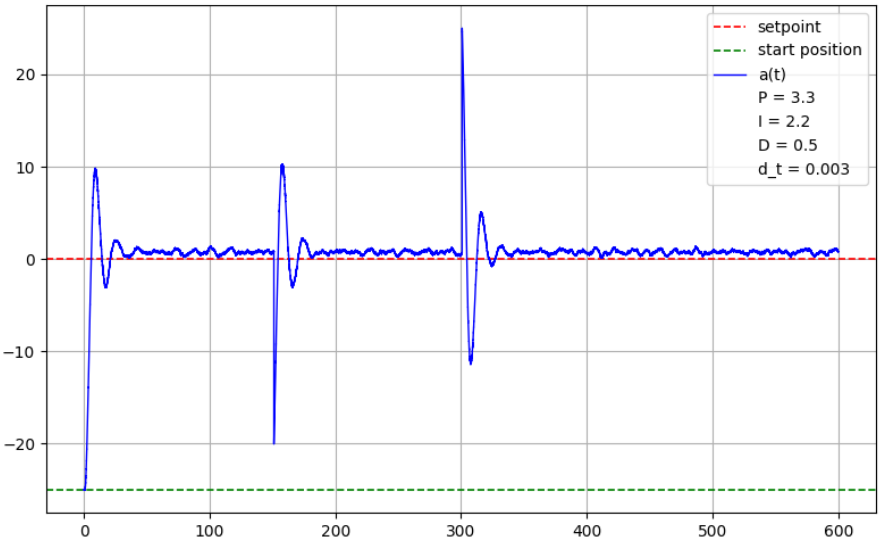




The analysis of the graph proves that the model is capable of describing the characteristics of the system and predicting its behavior with sufficient accuracy. Although some differences between the real and simulated data are visible, the general dynamic response of the system remains similar, which confirms the correctness of the developed mathematical model.

These graphs show the behavior of the system at the beginning of operation and its response to disturbances after stabilization, which is analogous to what was shown in the section 4. The system requires more time to stabilize, which is caused by the fact that friction was not taken into account. The presence of friction acts as an additional damping factor in the real system.

To improve the realism of the model, during each update of the error value in the main loop, a random value from the range of 0 to 1.5 is added to the new error value. This makes it possible to simulate the presence of noise in the system.



In this case the result looks more real and this behavior is similar to the results of real measurements, where the system did not reach complete stabilization at a specific position but continued to perform rapid oscillations around it.

5.3 Conclusions of the chapter

1) The mathematical model is capable of predicting the system behavior when correct initial parameters corresponding to the real characteristics of the system are used. Despite some differences in parameters such as reaction time and oscillation amplitude, the model provides results that are close to the real behavior of the system.
2) To refine the simulation results, an additional parameter can be introduced into the model to generate artificial noise in the measurements. The amplitude of this parameter variation should correspond to the amplitude of the sensor signal noise, which must be measured empirically.
3) Considering the real physical system as a universal one, the developed model can be applied to similar systems by adjusting the corresponding physical parameters and control coefficients.

6 Conclusions of the whole report

1) The developed propeller setup, consisting of a DC motor, counterweight mechanism, and MPU6050 sensor, provides a clear demonstration of the physical principles of balance and stabilization. The constructional design — combining 3D-printed elements with metal parts — ensures mechanical stability, modularity, and the ability to adjust the center of mass precisely for experimental purposes.
2) The implemented control system based on the Arduino Nano, BTS7960 driver, and MPU6050 sensor allows real-time tuning of the PID coefficients. The integration of encoder input, LCD visualization, and plug-in connections increases the system's usability and enables precise control of the propeller's dynamic response under variable conditions.
3) The experimental analysis confirmed the effectiveness of the PID algorithm in stabilizing the system and reducing oscillations. Each term of the controller — proportional, integral, and derivative — demonstrated its specific influence on system behavior: proportional gain ensured fast reaction, integral gain eliminated steady-state error, and derivative gain suppressed overshoot. Together, they provide a stable and efficient control performance suitable for further modeling, simulation, and advanced optimization research.
4) The developed mathematical model can be further integrated into a real-time simulation or control environment, allowing to test and optimize PID parameters before applying them to the physical system. This approach not only improves the precision of control but also reduces the risk of hardware wear during experimental tuning.

7 References

[1] https://limewire.com/d/BpuQO#ATQdRpJd1C

[2] https://www.crystalinstruments.com/blog/2020/8/23/pid-control-theory

[3] https://pastebin.com/CvDtehvy

[4] Y. Li, K. H. Ang, and G. C. Y. Chong, *PID Control System Analysis and Design: Problems, Remedies, and Future Directions*, *IEEE Control Systems Magazine*, vol. 26, no. 1, pp. 32–41, 2006.

[5] Q. Ariyansyah and A. Ma'arif, *DC Motor Speed Control with PID Control on the Prototype of a Mini-Submarine*, *Journal of Fuzzy Systems and Control*, vol. 1, no. 1, pp. 18–20, 2023.

[6] D. Bista, *Understanding and Design of an Arduino-Based PID Controller*, *Virginia Commonwealth University*, 2016.

[7] T. Allam, M. Raju, and S. Sundeep Kumar, *Design of PID Controller for DC Motor Speed Control Using Arduino Microcontroller*, *IRJET*, vol. 3, no. 9, pp. 791–793, 2016.

[8] https://pastebin.com/Xk4rEDF5